DOCKER

Guia de Referência — Comandos Essenciais

Para alunos iniciantes e intermediários

Este guia apresenta os principais comandos do Docker organizados por categoria. Utilize-o como material de consulta rápida durante seus estudos e projetos práticos.

1. Conceitos Básicos

Antes de usar os comandos, é importante entender os três pilares do Docker:

Imagem	Modelo/template somente-leitura que define o ambiente (SO, libs, app). Ex: ubuntu, nginx, node.
Container	Instância em execução de uma imagem. Isolado, mas compartilha o kernel do host.
Registry	Repositório de imagens. O Docker Hub (hub.docker.com) é o registry público padrão.

2. Gerenciamento de Imagens

2.1 Baixar, listar e remover imagens

Comando	Sintaxe / Exemplo	Descrição
`docker pull`	`docker pull nginx:latest`	Baixa uma imagem do Docker Hub
`docker images`	`docker images`	Lista todas as imagens locais
`docker rmi`	`docker rmi nginx:latest`	Remove uma imagem local
`docker image prune`	`docker image prune -a`	Remove todas as imagens não usadas

Comando	Sintaxe / Exemplo	Descrição
docker inspect	`docker inspect nginx`	Exibe detalhes completos de uma imagem
docker history	`docker history nginx`	Mostra as camadas de uma imagem
docker tag	`docker tag app:v1 meuusuario/app:v1`	Cria um alias/tag para uma imagem

Use tags específicas (ex: nginx:1.25) em vez de 'latest' para garantir reprodutibilidade.

3. Gerenciamento de Containers

3.1 Criar e executar containers

Comando	Sintaxe / Exemplo	Descrição
docker run	`docker run nginx`	Cria e inicia um container
docker run -d	`docker run -d nginx`	Executa em modo detached (background)
docker run -it	`docker run -it ubuntu bash`	Executa com terminal interativo
docker run --name	`docker run --name meu-nginx nginx`	Define nome para o container
docker run -p	`docker run -p 8080:80 nginx`	Mapeia porta host:container
docker run -v	`docker run -v /host:/app nginx`	Monta volume (bind mount)
docker run -e	`docker run -e SENHA=123 app`	Define variável de ambiente
docker run --rm	`docker run --rm ubuntu echo oi`	Remove container ao finalizar
docker run --network	`docker run --network minha-rede app`	Conecta a uma rede específica

3.2 Controlar containers em execução

Comando	Sintaxe / Exemplo	Descrição
docker ps	`docker ps`	Lista containers em execução
docker ps -a	`docker ps -a`	Lista todos os containers (inclusive parados)
docker start	`docker start meu-nginx`	Inicia um container parado
docker stop	`docker stop meu-nginx`	Para um container com graceful shutdown
docker kill	`docker kill meu-nginx`	Para um container forçadamente
docker restart	`docker restart meu-nginx`	Reinicia um container
docker pause	`docker pause meu-nginx`	Pausa os processos do container
docker unpause	`docker unpause meu-nginx`	Retoma os processos do container
docker rm	`docker rm meu-nginx`	Remove um container parado
docker rm -f	`docker rm -f meu-nginx`	Remove um container em execução
docker container prune	`docker container prune`	Remove todos containers parados

💡 docker stop é preferível a docker kill, pois permite ao processo finalizar corretamente.

3.3 Inspecionar e depurar containers

Comando	Sintaxe / Exemplo	Descrição
docker logs	`docker logs meu-nginx`	Exibe os logs do container
docker logs -f	`docker logs -f meu-nginx`	Acompanha os logs em tempo real
docker exec	`docker exec meu-nginx ls /`	Executa comando em container ativo
docker exec -it	`docker exec -it meu-nginx bash`	Abre terminal no container
docker inspect	`docker inspect meu-nginx`	Retorna JSON com todos os detalhes

Comando	Sintaxe / Exemplo	Descrição
docker stats	`docker stats`	Monitora uso de CPU/mem em tempo real
docker top	`docker top meu-nginx`	Lista processos rodando no container
docker diff	`docker diff meu-nginx`	Mostra alterações no filesystem
docker cp	`docker cp meu-nginx:/etc/nginx .`	Copia arquivos entre host e container

4. Criando Imagens com Dockerfile

O Dockerfile é um arquivo de texto com instruções para construir uma imagem personalizada.

4.1 Instruções mais usadas no Dockerfile

Comando	Sintaxe / Exemplo	Descrição
FROM	`FROM node:18-alpine`	Define a imagem base (obrigatório, 1º instrução)
RUN	`RUN apt-get install -y curl`	Executa comando durante o build
COPY	`COPY . /app`	Copia arquivos do host para a imagem
ADD	`ADD app.tar.gz /app`	Como COPY, mas suporta URLs e extração
WORKDIR	`WORKDIR /app`	Define o diretório de trabalho padrão
ENV	`ENV NODE_ENV=production`	Define variável de ambiente persistente
ARG	`ARG VERSAO=1.0`	Define argumento passado no build
EXPOSE	`EXPOSE 3000`	Documenta porta que o container usa
CMD	`CMD ["node", "app.js"]`	Comando padrão ao iniciar o container
ENTRYPOINT	`ENTRYPOINT ["npm","start"]`	Define executável principal do container

Comando	Sintaxe / Exemplo	Descrição
VOLUME	`VOLUME /data`	Declara ponto de montagem de volume
USER	`USER node`	Define usuário para executar os processos
LABEL	`LABEL version="1.0"`	Adiciona metadados à imagem
HEALTHCHECK	`HEALTHCHECK CMD curl -f localhost/`	Define verificação de saúde

4.2 Construir e publicar imagens

Comando	Sintaxe / Exemplo	Descrição
docker build	`docker build -t meuapp:1.0 .`	Constrói imagem a partir do Dockerfile
docker build -f	`docker build -f Dockerfile.prod .`	Usa Dockerfile específico
docker build --no-cache	`docker build --no-cache -t app .`	Build sem usar cache
docker push	`docker push usuario/meuapp:1.0`	Envia imagem ao Docker Hub
docker login	`docker login`	Autentica no Docker Hub
docker logout	`docker logout`	Encerra sessão no registry

> 💡 Use .dockerignore para excluir node_modules, .git e outros arquivos desnecessários do contexto de build.

5. Volumes e Persistência de Dados

Dados em containers são efêmeros por padrão. Volumes garantem persistência além do ciclo de vida do container.

Comando	Sintaxe / Exemplo	Descrição
docker volume create	`docker volume create meus-dados`	Cria um volume nomeado
docker volume ls	`docker volume ls`	Lista todos os volumes

Comando	Sintaxe / Exemplo	Descrição
docker volume inspect	`docker volume inspect meus-dados`	Detalhes de um volume
docker volume rm	`docker volume rm meus-dados`	Remove um volume
docker volume prune	`docker volume prune`	Remove volumes não utilizados
-v (named)	`docker run -v dados:/app/data img`	Monta volume nomeado
-v (bind)	`docker run -v $(pwd):/app img`	Monta diretório do host
--mount	`--mount type=volume,src=dados,dst=/data`	Sintaxe explícita e recomendada

💡 Prefira volumes nomeados a bind mounts em produção — são gerenciados pelo Docker e mais portáveis.

6. Redes (Networking)

Redes Docker permitem que containers se comuniquem entre si e com o mundo externo.

Comando	Sintaxe / Exemplo	Descrição
docker network ls	`docker network ls`	Lista as redes existentes
docker network create	`docker network create minha-rede`	Cria uma rede bridge
docker network inspect	`docker network inspect minha-rede`	Detalhes de uma rede
docker network rm	`docker network rm minha-rede`	Remove uma rede
docker network connect	`docker network connect rede container`	Conecta container à rede
docker network disconnect	`docker network disconnect rede container`	Desconecta container
docker network prune	`docker network prune`	Remove redes não utilizadas

Tipo de Rede	Descrição
bridge (padrão)	Rede privada interna. Containers se comunicam por IP ou nome (user-defined).

`host`	Container compartilha a rede do host. Sem isolamento de porta.
`none`	Container sem acesso à rede. Máximo isolamento.
`overlay`	Para comunicação entre hosts (Docker Swarm/clusters).

7. Docker Compose

O Docker Compose permite definir e gerenciar aplicações multi-container com um arquivo YAML (docker-compose.yml).

Comando	Sintaxe / Exemplo	Descrição
`docker compose up`	`docker compose up`	Cria e inicia todos os serviços
`docker compose up -d`	`docker compose up -d`	Inicia em modo background
`docker compose down`	`docker compose down`	Para e remove containers e redes
`docker compose down -v`	`docker compose down -v`	Remove também os volumes
`docker compose ps`	`docker compose ps`	Lista status dos serviços
`docker compose logs`	`docker compose logs -f web`	Exibe logs de um serviço
`docker compose build`	`docker compose build`	Reconstrói as imagens
`docker compose exec`	`docker compose exec web bash`	Executa comando em serviço
`docker compose restart`	`docker compose restart web`	Reinicia um serviço
`docker compose stop`	`docker compose stop`	Para os serviços sem remover
`docker compose pull`	`docker compose pull`	Baixa imagens atualizadas
`docker compose config`	`docker compose config`	Valida e exibe o compose.yml

💡 Use 'docker compose up --build' para forçar rebuild das imagens antes de iniciar.

8. Comandos de Sistema e Limpeza

Comando	Sintaxe / Exemplo	Descrição
docker info	`docker info`	Exibe informações do sistema Docker
docker version	`docker version`	Mostra versão do cliente e servidor
docker system df	`docker system df`	Uso de disco por imagens/containers/volumes
docker system prune	`docker system prune`	Remove todos recursos não utilizados
docker system prune -a	`docker system prune -a --volumes`	Limpeza completa (cuidado!)
docker events	`docker events`	Monitora eventos em tempo real
docker search	`docker search ubuntu`	Pesquisa imagens no Docker Hub

💡 Execute 'docker system prune' regularmente para liberar espaço em disco durante o desenvolvimento.

9. Referência Rápida — Cheat Sheet

Os atalhos e combinações mais usados no dia a dia:

Ação	Comando
Parar todos containers	`docker stop $(docker ps -q)`
Remover todos containers	`docker rm $(docker ps -aq)`
Remover todas imagens	`docker rmi $(docker images -q)`
Ver IP do container	`docker inspect -f '{{.NetworkSettings.IPAddress}}' nome`
Copiar arquivo para container	`docker cp arquivo.txt container:/destino/`
Acessar shell do container	`docker exec -it container sh`
Ver variáveis de ambiente	`docker exec container env`
Exportar container como imagem	`docker commit container nova-imagem:tag`
Build com arquivo específico	`docker build -f Dockerfile.dev -t app:dev .`
Rodar e remover ao sair	`docker run --rm -it ubuntu bash`